Universitas	: Universitas Logistik dan Bisnis Internasional (ULBI)
Program Studi	: D4 Teknik Informatika
Semester	: 5 (Lima)
Mata Kuliah	: Pemrograman IV (Pemrograman Mobile)
Topik	: Praktikum 4 Membuat Layout

Sebelum masuk ke praktikum kita akan coba membuat dasar-dasar layout, buat project dengan nama **latihanp4**, kita buat terlebih dahulu struktur Scaffold seperti di bawah. Buat file material_app.dart isi dengan kode di bawah

```dart
import 'package:flutter/material.dart';
import 'package:p3/material_page.dart';

class AppMaterial extends StatelessWidget {
  const AppMaterial({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      theme: ThemeData.light(useMaterial3: false),
      home: HomePage(),
    );
  }
}
```

Buat file material_page.dart isi dengan kode di bawah

```dart
import 'package:flutter/material.dart';

class HomePage extends StatelessWidget {
  const HomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Home Page"),
        backgroundColor: Colors.lightBlue,
      ),
      drawer: Drawer(
        child: ListView(
          children: [
            ListTile(title: Text('Homepage')),
            ListTile(title: Text('Settings')),
          ],
        ),
      ),
      // body: isiNamaFungsi(),
      bottomNavigationBar: BottomNavigationBar(
        items: const [
```

```
          BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),
          BottomNavigationBarItem(
            icon: Icon(Icons.settings),
            label: 'Settings',
          ),
        ],
      ),
    );
  }
}
```

Sekarang buka file main.dart lalu panggil class AppMaterial yang sudah dibuat

A. Pengenalan Container

Pada bagian sebelumnya kita mempelajari widget Scaffold. Kini saatnya belajar tentang widget Container. Container adalah widget yang digunakan untuk melakukan styling, membuat sebuah shape (bentuk), dan layout pada widget child-nya. Sebagai contoh buat di dalam file material_page.dart :

```
class MyContainer extends StatelessWidget {
  const MyContainer({super.key});

  @override
  Widget build(BuildContext context) {
    return Container(
      color: Colors.blue,
      child: const Text(
        'Hi',
        style: TextStyle(fontSize: 40),
      ),
    );
  }
}
```

Panggil class atau widget MyContainer pada class HomePage yang sudah dibuat saat pertemuan ke-3 yaitu di file material_page.dart

```
  body: MyContainer(),
```

Pada kode di atas kita membuat sebuah Text "Hi" yang dibungkus oleh widget Container dan kita beri property **color** dengan nilai **Colors.blue**.. Apa hasilnya? Text "Hi" akan memiliki background berwarna biru. Jalankan project untuk menampilkan hasil seperti berikut:

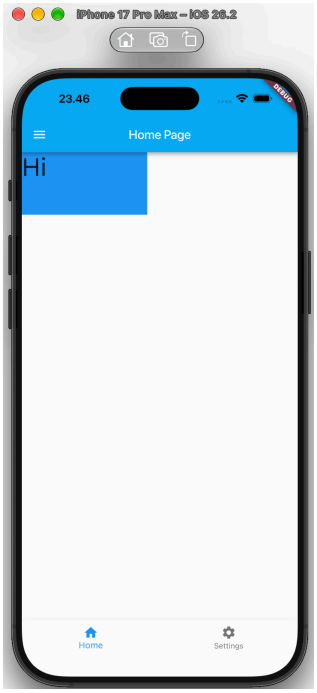


1. Width & Height

Kita dapat mengatur lebar (width) dan tinggi (height) suatu Container seperti berikut:

```
Container(
      color: Colors.blue,
      width: 200,
      height: 100,
      child: const Text(
        'Hi',
        style: TextStyle(fontSize: 40),
      ),
);
```

Kode di atas ketika dijalankan hasilnya akan seperti berikut:



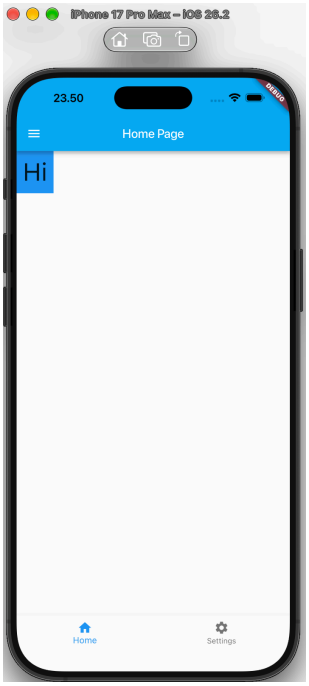
2. Padding & Margin

Container menyediakan padding & margin. Padding merupakan jarak antara konten (child) dengan Container, sedangkan margin merupakan jarak antara Container dengan bagian luar container.

Penggunaan padding adalah seperti berikut:

```
Container(
      color: Colors.blue,
      padding: const EdgeInsets.all(10),
      child: const Text(
        'Hi',
        style: TextStyle(fontSize: 40),
      ),
    );
```

Pada kode di atas kita menambahkan padding pada semua sisi container secara merata dengan nilai 10. Maka jika me-refresh aplikasi flutter, akan ada jarak antara Text "Hi" dengan batas (border) dari container.



Lalu penggunaan margin pun sama seperti halnya padding, maka contoh kodenya seperti berikut:

```
Container(
      color: Colors.blue,
      margin: const EdgeInsets.all(10),
      child: const Text(
        'Hi',
        style: TextStyle(fontSize: 40),
      ),
    );
```

Maka hasil dari kode di atas Container akan bergeser lebih ke dalam karena ada jarak antara Container dengan bagian luar Container.



B. Dekorasi Container

Decoration merupakan bagian dari Container untuk styling. Pada decoration kita dapat menentukan warna background (solid/gradient color), shadow, border, border radius (membulatkan sudut), mengatur shape (bentuk), dan lain-lain.

1. Color

Contoh menentukan warna background dari container dengan decoration seperti berikut:

```
Container(
      decoration: BoxDecoration(
        color: Colors.red,
      ),
      child: const Text(
        'Hi',
        style: TextStyle(fontSize: 40),
      ),
    );
```

Ketika dijalankan maka tampilan aplikasi akan seperti berikut:



Untuk menggunakan decoration cukup menambahkan property atau parameter decoration pada Container lalu beri nilai BoxDecoration. Pada contoh di atas kita merubah warna Container menjadi merah dengan memberi parameter color pada BoxDecoration. Ada catatan penting ketika menggunakan color pada BoxDecoration, yaitu pastikan tidak memberi parameter color pada Container.

2. **Shape**

Contoh selanjutnya pada decoration adalah kita akan mengatur shape (bentuk) dari Container, contohnya sebagai berikut:

```
Container(
      decoration: BoxDecoration(
        color: Colors.red,
        shape: BoxShape.circle,
      ),
      child: const Text(
        'Hi',
        style: TextStyle(fontSize: 40),
      ),
    );
```

Pada kode di atas kita menambahkan parameter shape dengan nilai BoxShape.circle. Artinya, bentuk dari Container tersebut akan berbentuk lingkaran. BoxShape memiliki opsi circle atau rectangle.



3. Shadow

Untuk menambahkan shadow pada Container kita akan menambahkan parameter boxShadow pada BoxDecoration, seperti berikut:

```
Container(
      decoration: BoxDecoration(
        color: Colors.red,
        boxShadow: const [
          BoxShadow(
            color: Colors.black,
            offset: Offset(3, 6),
            blurRadius: 10,
          ),
        ],
      ),
      child: const Text(
        'Hi',
        style: TextStyle(fontSize: 40),
      ),
    );
```

Pada kode di atas parameter boxShadow merupakan sebuah Array. Di dalamnya terdapat BoxShadow yang artinya pada Container kita dapat memberikan banyak bayangan atau shadow.



4. **Border**

Border merupakan batas garis dengan content (child). Begini cara menambahkan border pada container:

```
Container(
      decoration: BoxDecoration(
        color: Colors.red,
        border: Border.all(color: Colors.green, width: 3),
      ),
      child: const Text(
        'Hi',
        style: TextStyle(fontSize: 40),
      ),
    );
```



Apabila kalian ingin membuat border yang tidak berujung lancip cukup tambahkan parameter borderRadius Pada BoxDecoration seperti berikut:

```
Container(
      decoration: BoxDecoration(
        color: Colors.red,
        border: Border.all(color: Colors.green, width: 3),
        borderRadius: BorderRadius.circular(15),
      ),
      child: const Text(
        'Hi',
        style: TextStyle(fontSize: 40),
      ),
    );
```



Dengan menggunakan Widget Container kita dapat membuat variasi widget yang kita buat. Sebenarnya banyak sekali parameter-parameter yang dapat digunakan pada Container dan juga pada BoxDecoration. Kalian dapat mengeksplorasi hal tersebut dengan mencarinya di Google atau pada dokumentasi resmi flutter.

Container Class

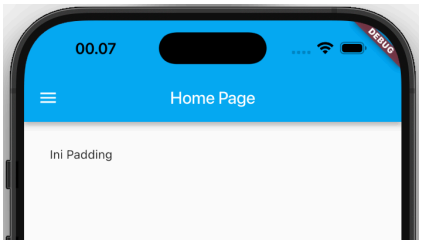
C. Padding

Sebelumnya kita telah belajar banyak hal mengenai Container. Kali ini kita akan belajar widget Padding. Seperti namanya widget Padding merupakan sebuah widget yang khusus untuk memberikan padding pada suatu widget.

Contoh penggunaan widget Padding seperti berikut:

```
Padding(
      padding: const EdgeInsets.all(30),
      child: const Text('Ini Padding'),
    );
```

Pada kode di atas widget Padding harus memiliki child. Child di sini merupakan sebuah widget yang nantinya akan diberi padding. Parameter padding ditambahkan untuk menentukan besaran padding yang diinginkan.



Kalian dapat membaca informasi detail tentang Padding pada dokumentasi berikut:

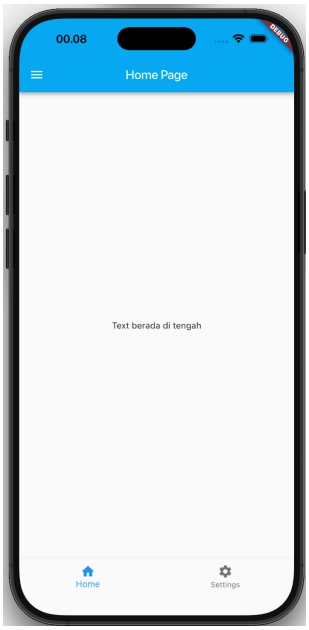
Padding Class

D. Center

Widget Center merupakan sebuah widget yang digunakan untuk membuat suatu widget berada pada posisi tengah. Penggunaan widget Center sangatlah simpel, yakni seperti berikut:

```
Center(
      child: const Text('Text berada di tengah'),
    );
```

Widget Center hanya membutuhkan parameter child untuk membuat widget di dalamnya berada pada posisi tengah. Hasil dari Center seperti berikut:



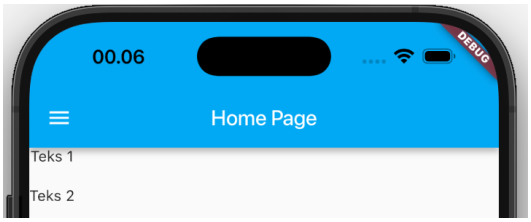
E. SizedBox

SizedBox adalah salah satu widget di Flutter yang digunakan untuk mengatur ukuran ruang kosong (jarak) dalam tata letak. Widget ini berguna untuk mengendalikan jarak antara elemen-elemen dalam tata letak, baik secara horizontal maupun vertikal. Berikut adalah beberapa poin penting tentang SizedBox:

- Mengatur Ukuran: SizedBox memungkinkan untuk mengatur ukuran lebar dan tinggi secara eksplisit. Kalian dapat mengatur ukuran widget ini sesuai kebutuhan.
- Mengatur Jarak: SizedBox digunakan untuk membuat jarak vertikal atau horizontal antara elemen-elemen dalam tata letak. Misalnya, kalian dapat menggunakan SizedBox untuk membuat spasi antara teks atau widget lainnya.
- Tidak Menggambar Apa-apa: Perlu dicatat bahwa SizedBox tidak menggambar elemen apa pun. Ini hanyalah sebuah widget yang memberikan ruang kosong sesuai dengan ukuran yang kalian tentukan.
- Contoh Penggunaan: Berikut adalah contoh penggunaan SizedBox untuk membuat spasi vertikal antara dua teks:

```
Column(
      children: <Widget>[
        Text('Teks 1'),
        SizedBox(height: 20), // Spasi vertikal sebesar 20
        Text('Teks 2'),
      ],
    );
```

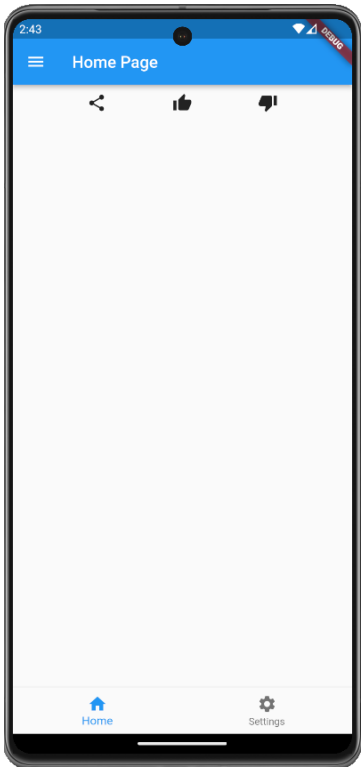
Dalam contoh di atas, SizedBox digunakan untuk membuat spasi vertikal sebesar 20 piksel antara "Teks 1" dan "Teks 2".



SizedBox adalah alat yang berguna untuk mengatur jarak dalam tata letak Flutter dengan cara yang sangat eksplisit dan mudah dipahami. Ini membantu dalam pengaturan tampilan yang lebih baik dan presisi dalam desain antarmuka pengguna.

F. Row dan Column

Selanjutnya kita akan mempelajari bagaimana cara membuat widget yang kita gunakan berjajar secara vertikal atau horizontal. Lalu apa yang dimaksud dengan membuat widget yang berjajar? Perhatikan gambar berikut:



Pada gambar di atas kita memiliki tampilan ikon-ikon yang merupakan kumpulan tombol, di antaranya share, like dan dislike. Tombol-tombol tersebut tersusun berjajar secara horizontal. Nah, untuk membuat berjajar horizontal atau membentuk baris kita menggunakan widget Row. Sedangkan untuk menyusun widget yang membentuk kolom atau vertikal, kita bisa menggunakan widget Column.

1. **Widget Column**

   Column merupakan suatu widget yang digunakan untuk membuat widget-widget tersusun berjajar secara vertikal. Column memiliki sintaks mirip dengan Row, seperti berikut:

```
Column(
  children: <Widget>[
    //di sini berisi widget-widget
  ]
)
```

   Contoh penerapan Column seperti berikut:

```
Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: <Widget>[
        Text(
          'Ini Judul',
          style: TextStyle(fontSize: 32, fontWeight: FontWeight.bold),
        ),
        Text('Universtitas Logistik dan Bisnis Internasional (ULBI)'),
```

```
      ],
    );
```

Di dalam widget Column menggunakan property crossAxisAlignment dengan nilai CrossAxisAlignment.start yang artinya teks di dalam Column menjadi rata kiri, penggunaan crossAxisAlignment merupakan hal yang opsional.



2. Widget Row

Widget Row merupakan suatu widget yang digunakan untuk membuat widget-widget tersusun berjajar secara horizontal. Row memiliki sintaks seperti berikut:

```
Row(
  children: <Widget>[
     //di sini berisi widget-widget
  ],
)
```

Untuk membuat widget-widget berjajar secara horizontal kita harus memasukkan widget-widget tersebut ke dalam parameter children. Parameter children berisi kumpulan atau list dari widget karena kita dapat menyusun beberapa widget sekaligus di dalamnya. Jika mengacu pada contoh tombol-tombol di atas kodenya seperti berikut:

```
Row(
  mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  children: <Widget>[
    Icon(Icons.share),
    Icon(Icons.thumb_up),
    Icon(Icons.thumb_down),
  ],
);
```

Seperti yang kita lihat, kita membuat sebuah IconButton berada di dalam parameter children. Kita menambahkan pula mainAxisAlignment yang merupakan parameter alignment pada Row. Parameter **mainAxisAlignment** yang berfungsi untuk mengatur alignment horizontal dari Row (alignment utama). Selain itu Row juga memiliki parameter crossAxisAlignment yang berfungsi untuk mengatur alignment secara vertikal. Kedua parameter ini juga berlaku sebaliknya untuk widget Column.



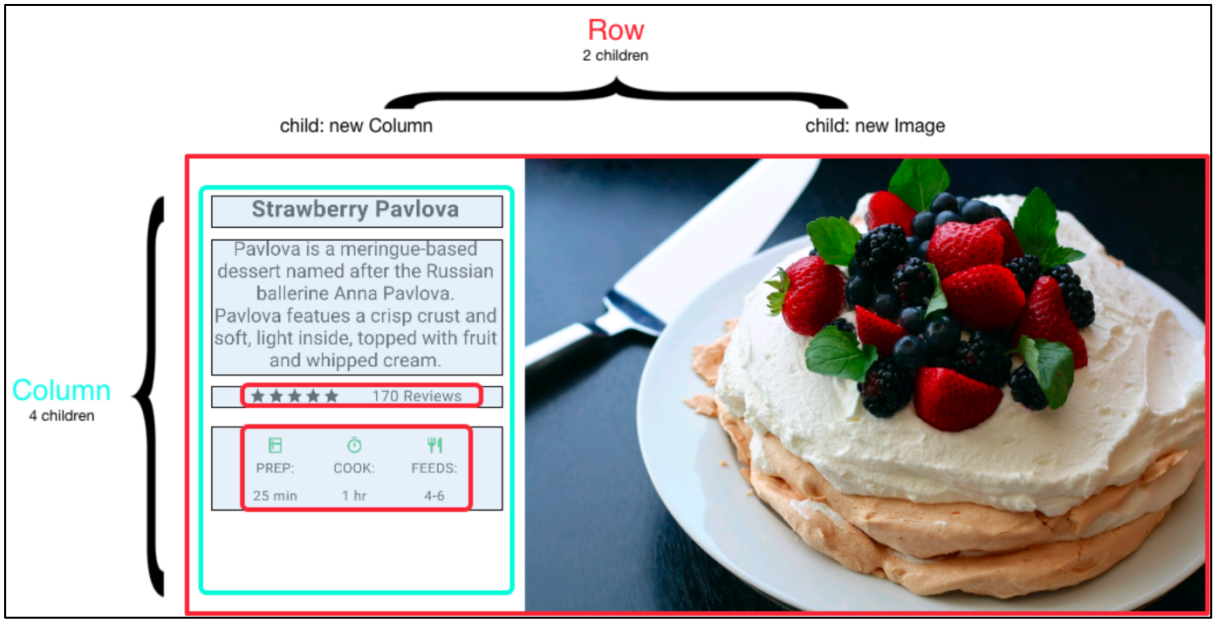
Berikut ini adalah contoh penerapan mainAxisAlignment pada Row:

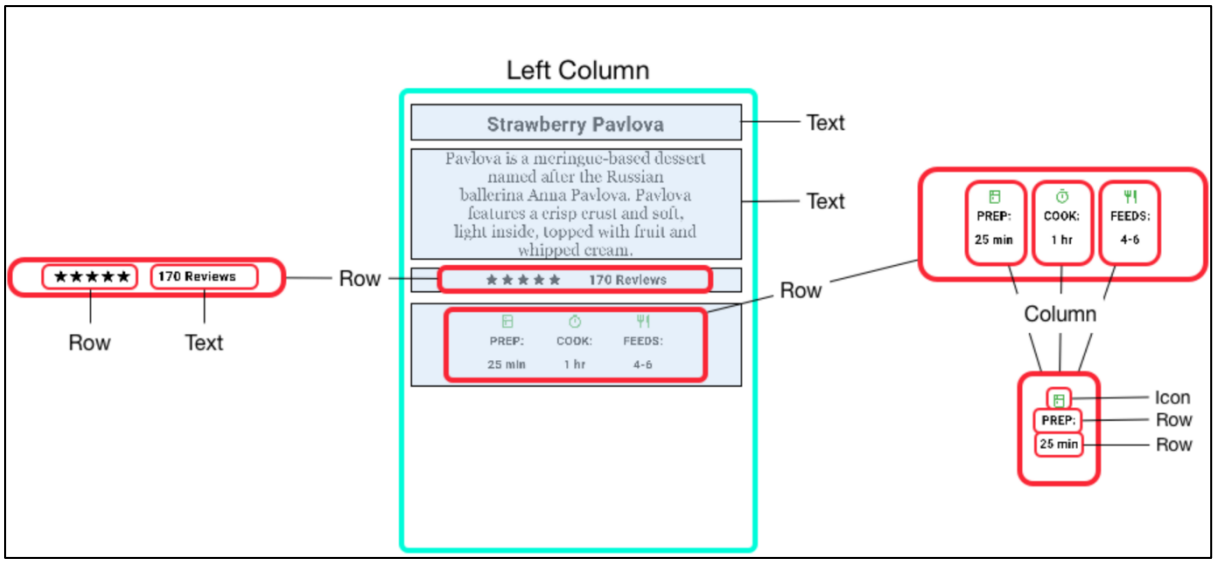


Task : Untuk latihan coba buatkan sebuah tampilan seperti gambar di atas

G. Kesimpulan

Untuk membuat sebuah widget-widget berjajar kita dapat menggunakan widget Row atau Column. Sebenarnya penggunaan Row dan Column dapat dipadukan sehingga dapat membuat sebuah layout yang kompleks seperti berikut:





Untuk memahami Row, Column, dan bagaimana menyusun layout dengan Flutter secara mendalam, silakan pelajari dokumentasi berikut:

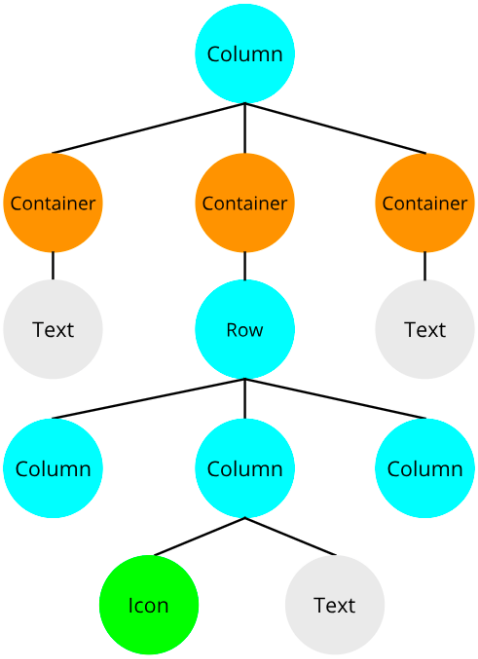
- Container Class
- Padding Class
- Row Class
- Column Class
- Layouts in Flutter

PERCOBAAN PRAKTIKUM

Dalam praktikum ini kita akan membuat sebuah tampilan yang menggabungkan semua widget-widget yang sebelumnya kita pelajari. Tampilannya adalah seperti berikut:



Sebelum kita membuat tampilan di atas, kita akan bedah terlebih dahulu layout-nya. Pada layout di atas kita dapat memetakan widget-widget dalam bentuk diagram seperti di bawah ini:



1. Buat project Flutter baru dan berikan nama **p4_1_npm (Misal p4_1_1214001)**. Hapus kode pada main.dart
2. Tuliskan kode dasar yang menampilkan widget MaterialApp seperti berikut dengan menggunakan StatelessWidget:

```dart
import 'package:flutter/material.dart';

Run | Debug | Profile
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(title: 'Tempat Wisata Bandung');
  }
}
```

3. Lalu kita akan membuat kode untuk susunan widget sesuai diagram yang telah kita buat. Untuk membuat kode kita lebih rapi kita akan membuat kelas Stateless Widget baru untuk menampung kode tampilan kita. Mari namakan kelas ini **DetailScreen**.

```dart
class DetailScreen extends StatelessWidget {
  const new({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold();
  }
}
```

Jangan lupa untuk menambahkan widget DetailScreen sebagai home dari MaterialApp.

```dart
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Tempat Wisata Bandung',
      home: const DetailScreen(),
    ); // MaterialApp
  }
}
```

4. Sesuai diagram di atas, kita akan menyusun beberapa widget secara vertikal sehingga kita perlu menggunakan widget Column.

```
class DetailScreen extends StatelessWidget {
  const new({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(body: Column());
  }
}
```

Jalankan aplikasi. Saat ini device atau emulator kalian memang masih belum menampilkan apa pun. Namun, kita akan memanfaatkan fitur hot reload untuk melihat perubahan-perubahan yang akan kita lakukan ke depan.

5. Komponen pertama yang akan kita buat adalah bagian judul dari halaman. Tentunya untuk menampilkan teks kita akan menggunakan widget Text.

```
class DetailScreen extends StatelessWidget {
  const new({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(body: Column(children: [const Text('Ranca Upas')]));
  }
}
```

6. Ketika kalian menyimpan project atau menjalankan hot reload, tampilan aplikasi sekarang mungkin tidak sesuai dengan keinginan, seperti teks terlalu ke atas dan juga terlalu kecil.



Untuk itulah kita perlu membungkus widget Text ke dalam Container supaya kita dapat memberikan property seperti margin atau padding.

7. Tambahkan margin atas supaya teks memiliki jarak terhadap bagian atas layar. Klik pada widget Text, kemudian klik kanan → Refactor → Wrap with Container.

```
class DetailScreen extends StatelessWidget {
  const new({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Column(
        children: [
          Container(
            margin: const EdgeInsets.only(top: 16.0),
            child: const Text('Ranca Upas'),
          ), // Container
        ],
      ), // Column
    ); // Scaffold
  }
}
```

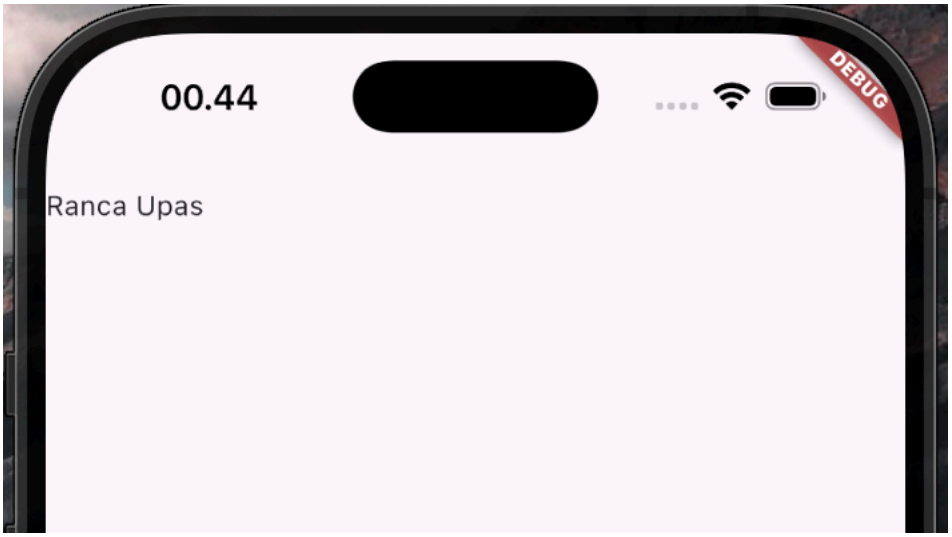
Pada kode di atas kita hanya memberikan margin atas sebesar sebesar 16.0. Kalian dapat memanfaatkan metode **EdgeInsets** lain seperti **all()** untuk memberikan margin ke semua sisi atau **symmetric()** apabila kalian ingin memberikan margin ke sisi vertikal atau horizontal.

8. Jika Anda kesulitan menentukan margin atas, khususnya pada perangkat yang memiliki notch yang umumnya memiliki status bar yang lebih besar, Anda dapat memanfaatkan widget **SafeArea**. Untuk membuat widget SafeArea dengan cara klik kiri pada widget Column, kemudian klik kanan → Refactor → Wrap with Widget → ubah nama widget menjadi SafeArea.

```dart
class DetailScreen extends StatelessWidget {
  const new({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: Column(
          children: [
            Container(
              margin: const EdgeInsets.only(top: 16.0),
              child: const Text('Ranca Upas'),
            ), // Container
          ],
        ), // Column
      ), // SafeArea
    ); // Scaffold
  }
}
```

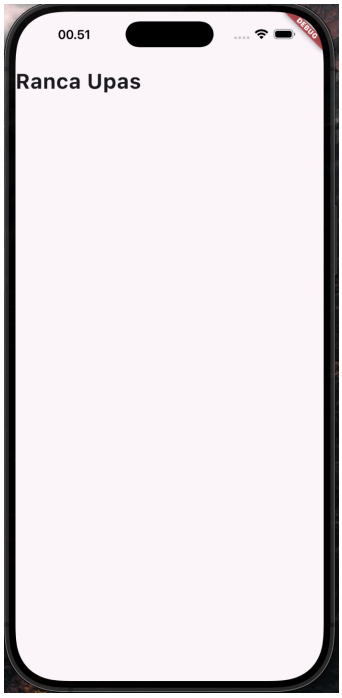
Widget ini akan memberikan padding yang secara otomatis menyesuaikan perangkat yang digunakan.



9. Selanjutnya, sesuai contoh kita akan membuat teks judul berada di tengah. Tambahkan parameter atau properti **textAlign** pada widget **Text**. Selain itu, tambahkan juga style dengan memperbesar ukuran teks agar tulisan dapat dibaca.

```dart
class DetailScreen extends StatelessWidget {
  const new({super.key});

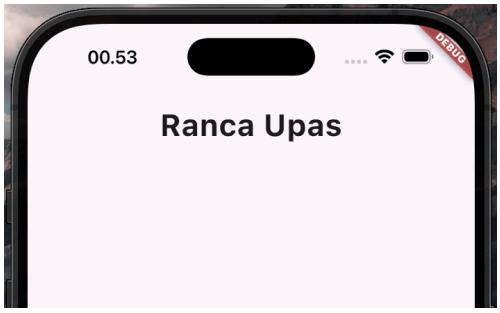
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: Column(
          children: [
            Container(
              margin: const EdgeInsets.only(top: 16.0),
              child: const Text(
                'Ranca Upas',
                textAlign: TextAlign.center,
                style: TextStyle(fontSize: 30.0, fontWeight: FontWeight.bold),
              ), // Text
            ), // Container
          ],
        ), // Column
      ), // SafeArea
    ); // Scaffold
  }
}
```



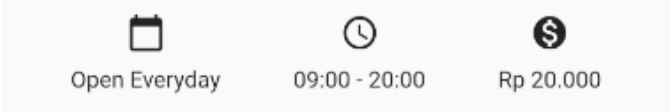
Dari gambar di atas bisa kita lihat ternyata layout aplikasi kita tidak penuh hingga seluruh halaman. Ini disebabkan sisi horizontal dari Column hanya menyesuaikan dengan konten yang ada di dalamnya. Untuk memaksimalkan ukuran lebar dari Column, tambahkan kode berikut:

```dart
class DetailScreen extends StatelessWidget {
  const DetailScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: <Widget>[
            Container(
              margin: const EdgeInsets.only(top: 16.0),
              child: const Text(
                'Ranca Upas',
                textAlign: TextAlign.center,
                style: TextStyle(
                  fontSize: 30.0,
                  fontWeight: FontWeight.bold,
                ), // TextStyle
              ), // Text
            ), // Container
          ], // <Widget>[]
        ), // Column
      ), // SafeArea
    ); // Scaffold
  }
}
```



10. Setelah menyelesaikan judul, selanjutnya kita akan membuat bagian kedua yaitu informasi dari tempat wisata.



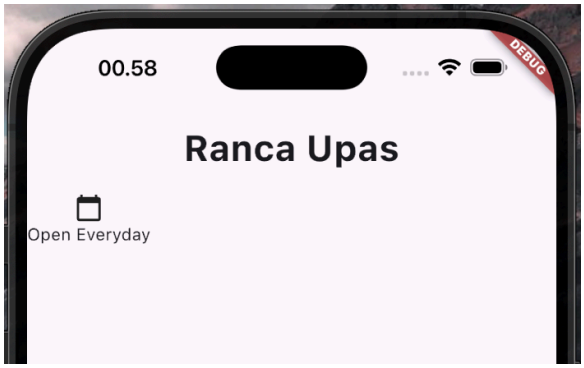
Seperti yang terlihat kita perlu menyusun widget secara horizontal dan vertikal. Mari tambahkan child kedua dari Column dengan sebuah Container berisi Row. Tambahkan juga margin pada sisi atas dan bawah untuk memberikan jarak antar widget.

Langkah pertama buat Container baru di bawah Container dari Judul yang sudah dibuat.

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    body: SafeArea(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Container(
            margin: const EdgeInsets.only(top: 16.0),
            child: const Text(
              'Ranca Upas',
              textAlign: TextAlign.center,
              style: TextStyle(fontSize: 30.0, fontWeight: FontWeight.bold),
            ), // Text
          ), // Container
          Container(
            margin: const EdgeInsets.symmetric(vertical: 16.0),
            child: Row(children: []),
          ), // Container
        ],
      ), // Column
    ), // SafeArea
  ); // Scaffold
}
```

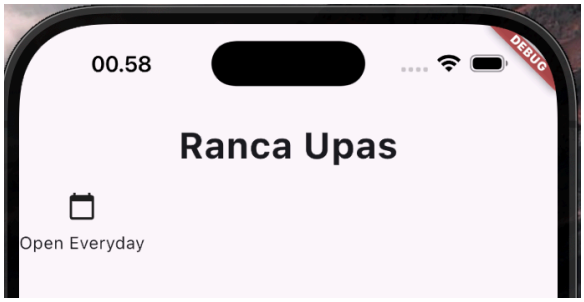
11. Tambah widget Column di dalam widget Row untuk menyusun Icon dan Text menjadi seperti di bawah.

```
Container(
  margin: const EdgeInsets.symmetric(vertical: 16.0),
  child: Row(
    children: [
      Column(
        children: const [
          Icon(Icons.calendar_today),
          Text('Open Everyday'),
        ],
      ), // Column
    ],
  ), // Row
), // Container
```

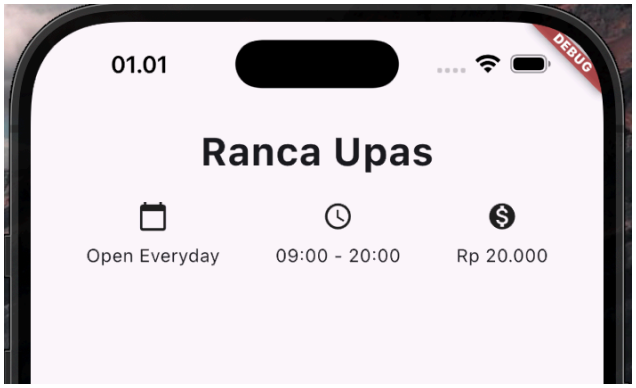


Jika kalian merasa jarak antara Icon dan Text terlalu rapat, Anda dapat menambahkan widget SizedBoxuntuk membuat “kotak” yang berguna untuk memberikan jarak.

```
Container(
  margin: const EdgeInsets.symmetric(vertical: 16.0),
  child: Row(
    children: [
      Column(
        children: const [
          Icon(Icons.calendar_today),
          SizedBox(height: 8.0),
          Text('Open Everyday'),
        ],
      ), // Column
    ],
  ), // Row
), // Container
```



12. **Selanjutnya sebagai tantangan, lengkapilah informasi tempat wisata dengan pasangan ikon dan teks sesuai contoh yang diberikan.**



Untuk menyusun Row seperti di atas, pastikan menggunakan **mainAxisAlignment.**

Iconnya : access_time dan monetization_on.

13. Pada langkah ini harusnya kalian sudah bisa menampilkan teks deskripsi sesuai langkah yang diberikan sebelumnya. Selanjutnya kalian tinggal menambahkan widget **Container** dan **Text** untuk menampilkan konten deskripsi. Anda juga dapat menambahkan style sesuai selera kalian.

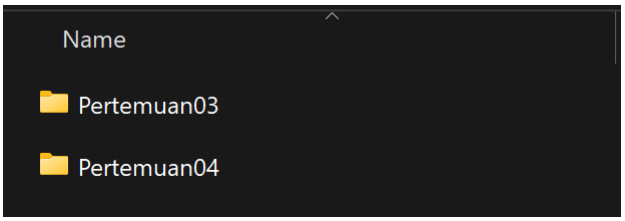
```
Container(
  padding: const EdgeInsets.all(16.0),
  child: const Text(
   'Ranca Upas Ciwidey adalah kawasan bumi perkemahan di bawah pengelolaan
perhutani. Tempat ini berada di kawasan wisata Bandung Selatan, satu lokasi
dengan kawah putih, kolam Cimanggu dan situ Patenggang. Banyak hal yang
bisa dilakukan di kawasan wisata ini, seperti berkemah, berinteraksi dengan
rusa, sampai bermain di water park dan mandi air panas.',
    textAlign: TextAlign.justify,
    style: TextStyle(fontSize: 16.0),
  ),
),
```

14. Horeee, saat ini kalian berhasil menyusun widget dengan baik. Tampilan aplikasi akan seperti gambar berikut.

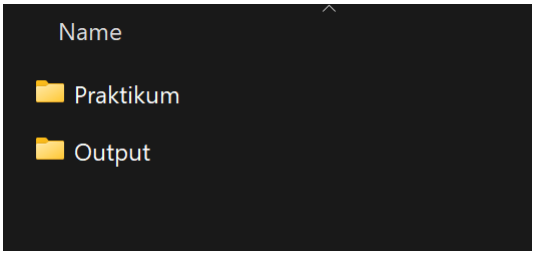


Pengumpulan Hasil :

Jika sudah berhasil, hasil praktikum dimasukkan ke dalam github masing-masing. Gunakan repository bernama PemrogramanIV_NPM



Karena sekarang adalah Pertemuan ke-4 maka pada folder **Pertemuan04** buatlah folder Praktikum dan Output



Isi folder praktikum adalah aplikasi flutter yang sudah dibuat dengan nama **p4_1_NPM**, tetapi **sebelum melakukan push ke github** wajib jalankan perintah **flutter clean** pada VSCode. Isi folder Output adalah hasil dari aplikasi flutter yang sudah dibuat (bisa berupa mp4, gif, png tapi disarankan dalam format gif).

Setelah melakukan perintah **flutter clean** kalian bisa langsung push ke dalam repository kalian.